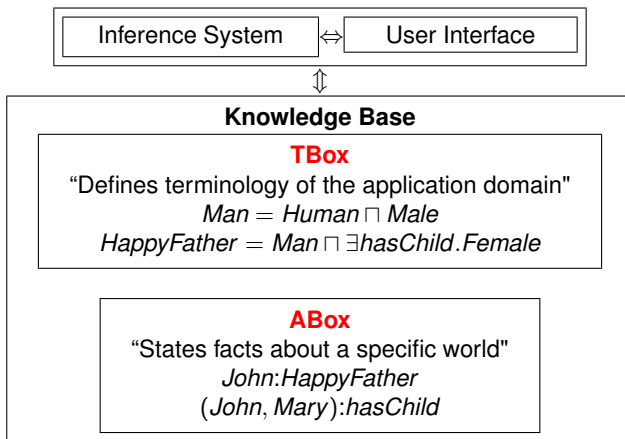


Description Logic Reasoning

Description Logic Reasoning

- What can we do with it?
 - ▶ **Design and maintenance** of ontologies
 - ★ Check class consistency and compute class hierarchy
 - ★ Particularly important with large ontologies/multiple authors
 - ▶ **Integration** of ontologies
 - ★ Assert inter-ontology relationships
 - ★ Reasoner computes integrated class hierarchy/consistency
 - ▶ **Querying** class and instance data w.r.t. ontologies
 - ★ Determine if set of facts are consistent w.r.t. ontologies
 - ★ Determine if individuals are instances of ontology classes
 - ★ Retrieve individuals/tuples satisfying a query expression
 - ★ Check if one class subsumes (is more general than) another w.r.t. ontology
- How do we do it?
 - ▶ Use DLs reasoner ($\text{OWL DL} = \text{SHOIN}(\mathcal{D})$)
 - ★ **Formal properties** well understood (complexity, decidability)
 - ★ Known practical **reasoning algorithms**
 - ★ **Implemented systems** (highly optimised)

Description Logic System



Basic Inference Problems (Formally)

Consistency: Check if knowledge is meaningful

- Is $\mathcal{K}$ satisfiability? $\mapsto$ Is there some model $\mathcal{I}$ of $\mathcal{K}$?
- Is C satisfiability? $\mapsto C^{\mathcal{I}} \neq \emptyset$ for some model $\mathcal{I}$ of $\mathcal{K}$?

Subsumption: structure knowledge, compute taxonomy

- $\mathcal{K} \models C \sqsubseteq D$? $\mapsto$ Is it true that $C^{\mathcal{I}} \subseteq D^{\mathcal{I}}$ for all models $\mathcal{I}$ of $\mathcal{K}$?

Equivalence: check if two classes denote same set of instances

- $\mathcal{K} \models C = D$? $\mapsto$ Is it true that $C^{\mathcal{I}} = D^{\mathcal{I}}$ for all models $\mathcal{I}$ of $\mathcal{K}$?

Instantiation: check if individual a instance of class C

- $\mathcal{K} \models a:C$? $\mapsto$ Is it true that $a^{\mathcal{I}} \in C^{\mathcal{I}}$ for all models $\mathcal{I}$ of $\mathcal{K}$?

Retrieval: retrieve set of individuals that instantiate C

- Compute the set $\{a \mid \mathcal{K} \models a:C\}$

Reduction to Satisfiability

Problems are all **reducible** to KB satisfiability

Subsumption: $\mathcal{K} \models C \sqsubseteq D$ iff $\langle \mathcal{T}, \mathcal{A} \cup \{a:C \sqcap \neg D\} \rangle$ not satisfiable,
where a is a new individual

Equivalence: $\mathcal{K} \models C = D$ iff $\mathcal{K} \models C \sqsubseteq D$ and $\mathcal{K} \models D \sqsubseteq C$

Instantiation: $\mathcal{K} \models a:C$ iff $\langle \mathcal{T}, \mathcal{A} \cup \{a:\neg C\} \rangle$ not satisfiable

Retrieval: The computation of the set $\{a \mid \mathcal{K} \models a:C\}$ is reducible to the instance checking problem

Reasoning in DLs: Basics

- Tableaux algorithm deciding concept satisfiability: is C satisfiable?, i.e., is there an interpretation $\mathcal{I}$ such that $C^{\mathcal{I}} \neq \emptyset$?
- Try to build a **tree-like model** $\mathcal{I}$ of the input concept C
- Decompose C syntactically
 - ▶ Apply tableau **expansion rules**
 - ▶ Infer constraints on elements of model
- Tableau rules correspond to constructors in logic ($\sqcap, \sqcup, \dots$)
 - ▶ Some rules are **nondeterministic** (e.g., $\sqcup, \leq$)
 - ▶ In practice, this means **search**
- Stop when no more rules applicable or **clash** occurs
 - ▶ Clash is an obvious contradiction, e.g., $A(x), \neg A(x)$
- Cycle check (**blocking**) may be needed for termination
- C satisfiable iff rules can be applied such that a fully expanded clash free tree is constructed

Tableaux checking satisfiability of an $\mathcal{ALC}$ concept

- Works on a tree (semantics through viewing tree as an ABox)
 - ▶ Nodes represent elements of $\Delta^{\mathcal{I}}$, labelled with sub-concepts of C
 - ▶ Edges represent role-successorships between elements of $\Delta^{\mathcal{I}}$
- Works on concepts in **negation normal form**: push negation inside using de Morgan' laws

$$\begin{array}{lll} \neg \top & \mapsto & \perp \\ \neg \perp & \mapsto & \top \\ \neg \neg C & \mapsto & C \\ \neg(C_1 \sqcap C_2) & \mapsto & \neg C_1 \sqcup \neg C_2 \\ \neg(C_1 \sqcup C_2) & \mapsto & \neg C_1 \sqcap \neg C_2 \end{array}$$

and

$$\begin{array}{lll} \neg(\exists R.C) & \mapsto & \forall R.\neg C \\ \neg(\forall R.C) & \mapsto & \exists R.\neg C \end{array}$$

- It is initialised with a tree consisting of a single (root) node x_0 with $\mathcal{L}(x_0) = \{C\}$
- A tree T contains a **clash** if, for a node x in T , $\{A, \neg A\} \subseteq \mathcal{L}(x)$
- Returns “ C is consistent” if rules can be applied s.t. they yield a clash-free, complete (no more rules apply) tree

$\mathcal{ALC}$ Tableau rules

$x \bullet \{C_1 \sqcap C_2, \dots\}$	$\longrightarrow \sqcap$	$x \bullet \{C_1 \sqcap C_2, \textcolor{red}{C}_1, \textcolor{red}{C}_2, \dots\}$
$x \bullet \{C_1 \sqcup C_2, \dots\}$	$\longrightarrow \sqcup$	$x \bullet \{C_1 \sqcup C_2, \textcolor{red}{C}, \dots\}$ for $C \in \{C_1, C_2\}$
$x \bullet \{\exists R.C, \dots\}$	$\longrightarrow \exists$	$x \bullet \{\exists R.C, \dots\}$ $\textcolor{red}{R} \downarrow$ $y \bullet \{\textcolor{red}{C}\}$
$x \bullet \{\forall R.C, \dots\}$ $R \downarrow$ $y \bullet \{\dots\}$	$\longrightarrow \forall$	$x \bullet \{\exists R.C, \dots\}$ $R \downarrow$ $y \bullet \{\dots, \textcolor{red}{C}\}$

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D\}$$

x

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D\}$$

x

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D\}$$

x

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

x

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

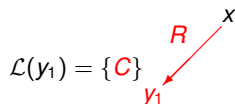
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

x

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

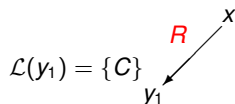
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

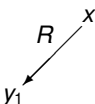
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

$$\mathcal{L}(y_1) = \{C, \neg C \sqcup \neg D\}$$


Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

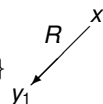
$$\mathcal{L}(y_1) = \{C, \neg C \sqcup \neg D\}$$


A diagram showing a directed edge from node x to node y_1 labeled R . The edge is a diagonal line with an arrowhead pointing towards y_1 .

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

$$\mathcal{L}(y_1) = \{C, \neg C \sqcup \neg D, \neg C\}$$


Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

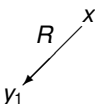
$$\mathcal{L}(y_1) = \{\textcolor{red}{C}, \neg C \sqcup \neg D, \textcolor{red}{\neg C}\}$$


Clash

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

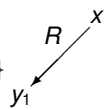
$$\mathcal{L}(y_1) = \{C, \neg C \sqcup \neg D\}$$


A diagram showing a directed edge from node x to node y_1 labeled R . The edge is a diagonal line with an arrow pointing from x to y_1 .

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

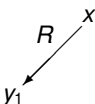
$$\mathcal{L}(y_1) = \{C, \neg C \sqcup \neg D, \neg D\}$$


A diagram showing a directed edge from node x to node y_1 labeled R . The edge is a diagonal line with an arrowhead pointing towards y_1 .

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$

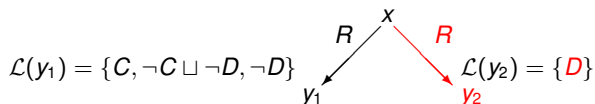
$$\mathcal{L}(y_1) = \{C, \neg C \sqcup \neg D, \neg D\}$$


A diagram showing a directed edge from node x to node y_1 labeled R . The edge is represented by a line with an arrow pointing from x to y_1 , with the label R placed above the line.

Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

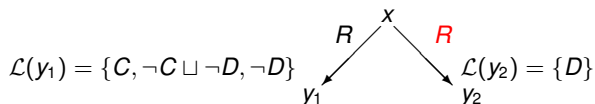
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

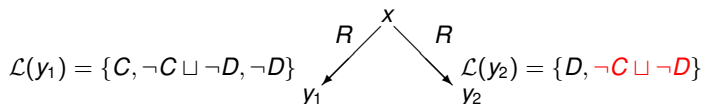
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

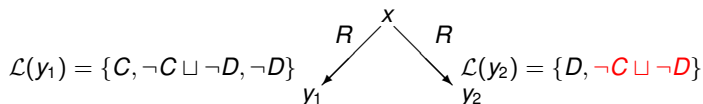
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

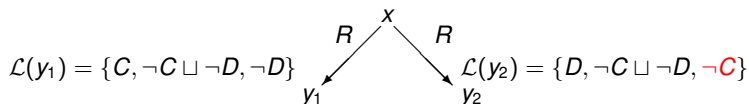
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

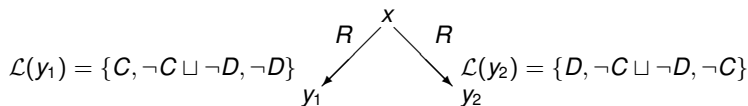
$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



Example

Is $\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D$ satisfiable? Yes.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.(\neg C \sqcup \neg D), \exists R.D\}$$



- Finished. No more rules applicable and the tableau is complete and clash-free
- Hence, the concept is **satisfiable**
- The tree corresponds to a **model** $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$
 - ▶ The nodes are the elements of the domain: $\Delta^{\mathcal{I}} = \{x, y_1, y_2\}$
 - ▶ For each atomic concept A , set $A^{\mathcal{I}} = \{z \mid A \in \mathcal{L}(z)\}$
 - ★ $C^{\mathcal{I}} = \{y_1\}$, $D^{\mathcal{I}} = \{y_2\}$
 - ▶ For each role R , set $R^{\mathcal{I}} = \{\langle x, y \rangle \mid \text{there is an edge labeled } R \text{ from } x \text{ to } y\}$
 - ★ $R^{\mathcal{I}} = \{\langle x, y_1 \rangle, \langle x, y_2 \rangle\}$
 - ▶ It can be shown that $x \in (\exists R.C \sqcap \forall R.(\neg C \sqcup \neg D) \sqcap \exists R.D)^{\mathcal{I}} \neq \emptyset$

Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C \sqcap \forall R.\neg C\}$$

x

Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C \sqcap \forall R.\neg C\}$$

x

Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C \sqcap \forall R.\neg C\}$$

x

Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.\neg C\}$$

x

Example

Is $\text{some}R.C \sqcap \forall R.\neg C$ satisfiable? No.

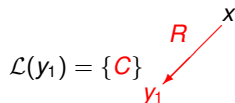
$$\mathcal{L}(x) = \{\exists R.C, \forall R.\neg C\}$$

x

Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

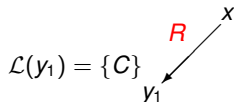
$$\mathcal{L}(x) = \{\exists R.C, \forall R.\neg C\}$$



Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.\neg C\}$$

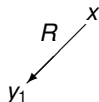


Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.\neg C\}$$

$$\mathcal{L}(y_1) = \{C, \neg C\}$$



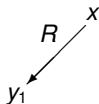
Example

Is $\exists R.C \sqcap \forall R.\neg C$ satisfiable? No.

$$\mathcal{L}(x) = \{\exists R.C, \forall R.\neg C\}$$

$$\mathcal{L}(y_1) = \{C, \neg C\}$$

Clash



- Finished. No more rules applicable and the tableau is complete, but **not** clash-free
- Hence, the concept is **not satisfiable**
- I.e. no model can be built, e.g.
 - $\Delta^{\mathcal{I}} = \{x, y_1\}$
 - $C^{\mathcal{I}} = \{y_1\}$
 - $R^{\mathcal{I}} = \{\langle x, y_1 \rangle\}$
 - is not a model because

$$(\exists R.C \sqcap \forall R.\neg C)^{\mathcal{I}} = (\exists R.C)^{\mathcal{I}} \cap (\forall R.\neg C)^{\mathcal{I}} = \{x\} \cap \emptyset = \emptyset$$

Soundness and Completeness

Theorem

Let C be an $\mathcal{ALC}$ concept and T the tree obtained by applying the tableau rules to C . Then

- 1 The rule application terminates,*
- 2 If T is clash-free and complete, then T defines a (canonical) (tree) model for C , and*
- 3 If C has a model $\mathcal{I}$, then the rules can be applied such that they yield a clash-free and complete T .*

Corollary

- 1 The tableau algorithm is a PSPACE (using depth-first search) decision procedure for the satisfiability (and subsumption) problem of $\mathcal{ALC}$ concepts.*
- 2 $\mathcal{ALC}$ has the tree-model property*

ABox Satisfiability

- We have seen how to test the satisfiability of a concept C ,
 - ▶ we start with the assertion $x:C$
 - ▶ create a tree with node x and node label $\mathcal{L}(x) = \{C\}$
 - ▶ apply the tableau rules until no more rule is applicable
 - ▶ and check if the tree is clash free and, thus, the concept is satisfiable. Otherwise, the concept is not satisfiable
- But, how can we check if an ABox

$$\mathcal{A} = \{a_1:C_1, \dots, a_n:C_n\} \cup \{(b_{1_1}, b_{1_2}):R_1, \dots, (b_{k_1}, b_{k_2}):R_k\}$$

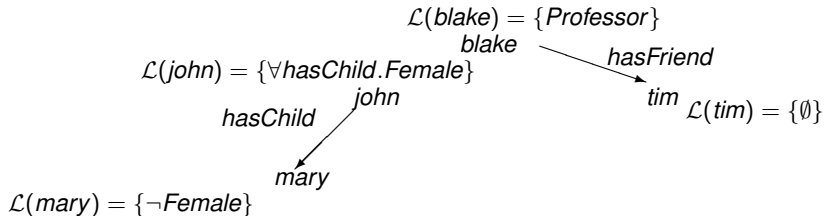
is satisfiable?

- ▶ rather than a tree, we use a **forest** of trees
- ▶ we create a node c for each individual $c \in \mathcal{A}$ with $\mathcal{L}(a_i) := \emptyset$
- ▶ for each $a:C \in \mathcal{A}$, we set $\mathcal{L}(a) := \mathcal{L}(a) \cup \{C\}$
- ▶ for each $(a, b):R \in \mathcal{A}$, we create an edge from a to b labeled R
- ▶ apply the tableau rules until no more rule is applicable
- ▶ and check if the forest is clash free and, thus, the Abox $\mathcal{A}$ is satisfiable. Otherwise, $\mathcal{A}$ is not satisfiable

Example

“Assume that all children of John are female, and that Mary is a child of John. Tim is a friend of professor Blake. Prove that Mary is female.”

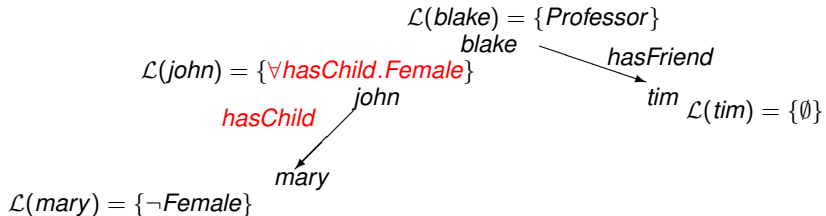
- Let $\mathcal{A} = \{john:\forall hasChild.Female, (john, mary):hasChild, (blake, tim):hasFriend, blake:Professor\}$
- We have to prove that $\mathcal{A} \models mary:Female$
- That is, we have to check whether $\mathcal{A} \cup \{mary:\neg Female\}$ is not satisfiable



Example

“Assume that all children of John are female, and that Mary is a child of John. Tim is a friend of professor Blake. Prove that Mary is female.”

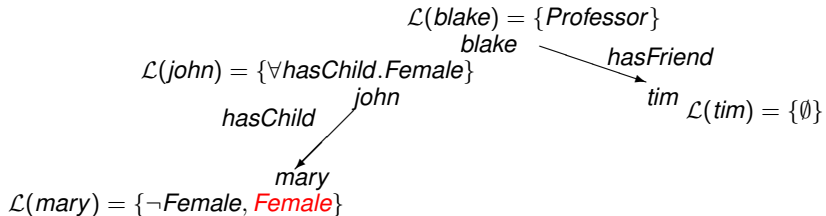
- Let $\mathcal{A} = \{john:\forall hasChild.Female, (john, mary):hasChild, (blake, tim):hasFriend, blake:Professor\}$
- We have to prove that $\mathcal{A} \models mary:Female$
- That is, we have to check whether $\mathcal{A} \cup \{mary:\neg Female\}$ is not satisfiable



Example

“Assume that all children of John are female, and that Mary is a child of John. Tim is a friend of professor Blake. Prove that Mary is female.”

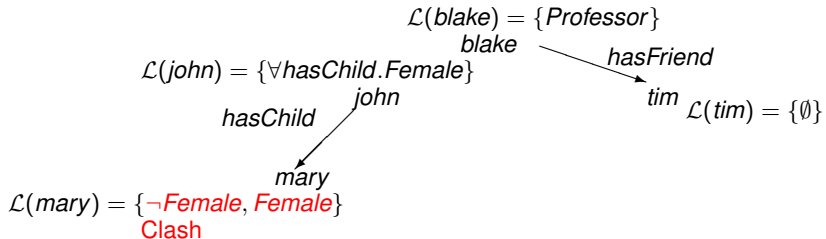
- Let $\mathcal{A} = \{john:\forall hasChild.Female, (john, mary):hasChild, (blake, tim):hasFriend, blake:Professor\}$
- We have to prove that $\mathcal{A} \models mary:Female$
- That is, we have to check whether $\mathcal{A} \cup \{mary:\neg Female\}$ is not satisfiable



Example

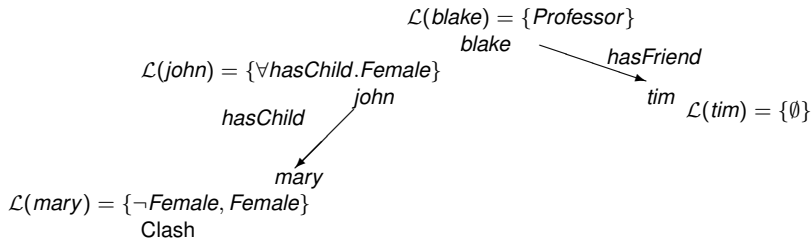
“Assume that all children of John are female, and that Mary is a child of John. Tim is a friend of professor Blake. Prove that Mary is female.”

- Let $\mathcal{A} = \{john:\forall hasChild.Female, (john, mary):hasChild, (blake, tim):hasFriend, blake:Professor\}$
- We have to prove that $\mathcal{A} \models mary:Female$
- That is, we have to check whether $\mathcal{A} \cup \{mary:\neg Female\}$ is not satisfiable



Example

- Let $\mathcal{A} = \{john:\forall hasChild.Female, (john, mary):hasChild, (blake, tim):hasFriend, blake:Professor\}$
- We have to prove that $\mathcal{A} \models mary:Female$
- That is, we have to check whether $\mathcal{A} \cup \{mary:\neg Female\}$ is not satisfiable



- Finished. No more rules applicable and the tableau is complete, but **not** clash-free
- No model can be built, e.g.
 - $\Delta^{\mathcal{I}} = \{john, mary, blake, tim\}$
 - $Female^{\mathcal{I}} = \{mary\}, Professor^{\mathcal{I}} = \{blake\}$
 - $hasChild^{\mathcal{I}} = \{\langle john, mary \rangle\}, hasFriend^{\mathcal{I}} = \{\langle blake, tim \rangle\}$
 - is not a model of $\mathcal{A} \cup \{mary:\neg Female\}$ because

$$mary \notin (\neg Female)^{\mathcal{I}} = \Delta^{\mathcal{I}} \setminus Female^{\mathcal{I}} = \{john, mary, blake, tim\} \setminus \{mary\} = \{john, blake, tim\}$$

- Hence, $\mathcal{A} \cup \{mary:\neg Female\}$ is **not satisfiable**, i.e., $\mathcal{A} \models mary:Female$

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\mathcal{T} = \{Human \sqsubseteq \exists hasMother.Human\}$$
$$\mathcal{A} = \{umberto:Human\}$$
$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human\}$$
$$umberto$$

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\mathcal{T} = \{Human \sqsubseteq \exists hasMother.Human\}$$

$$\mathcal{A} = \{umberto:Human\}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human\}$$

umberto

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\mathcal{T} = \{Human \sqsubseteq \exists hasMother.Human\}$$

$$\mathcal{A} = \{umberto:Human\}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human\} \quad umberto$$

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\mathcal{T} = \{Human \sqsubseteq \exists hasMother.Human\}$$

$$\mathcal{A} = \{umberto:Human\}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \neg Human\} \quad umberto$$

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\mathcal{T} = \{Human \sqsubseteq \exists hasMother.Human\}$$
$$\mathcal{A} = \{umberto:Human\}$$
$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \neg Human\}$$

umberto

Clash

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human\}$$

umberto

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\} \quad umberto$$

KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\} \quad umberto$$

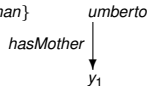
KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

$$\mathcal{L}(y_1) = \{\textcolor{red}{Human}, \neg \textcolor{red}{Human} \sqcup \textcolor{red}{\exists hasMother.Human}\}$$



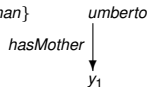
KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

$$\mathcal{L}(y_1) = \{Human, \neg Human \sqcup \exists hasMother.Human\}$$



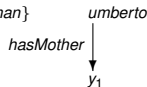
KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

$$\mathcal{L}(y_1) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$



KB Satisfiability

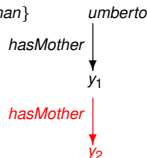
- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

$$\mathcal{L}(y_1) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

$$\mathcal{L}(y_2) = \{\textcolor{red}{Human}, \neg \textcolor{red}{Human} \sqcup \exists \textcolor{red}{hasMother.Human}\}$$



KB Satisfiability

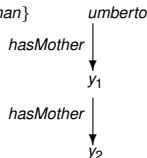
- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$

$$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

$$\mathcal{L}(y_1) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$$

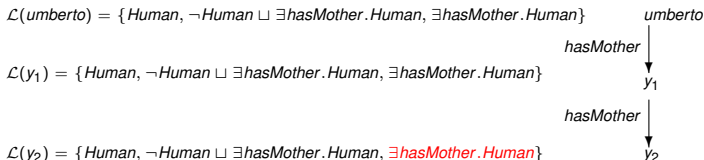
$$\mathcal{L}(y_2) = \{Human, \neg Human \sqcup \exists hasMother.Human\}$$



KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

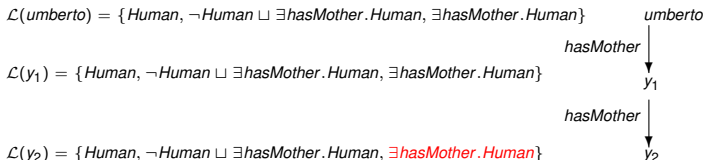
$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$



KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

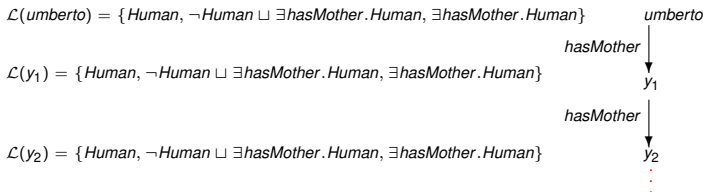
$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$



KB Satisfiability

- We have seen how to test the satisfiability of an ABox $\mathcal{A}$
- But, how can we check if a KB $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$ is satisfiable?
- Basic idea: since $t(C \sqsubseteq D) \equiv \forall x. \neg t(C, x) \vee t(D, x)$
 - ▶ we use the rule: for each $C \sqsubseteq D \in \mathcal{T}$, add $\neg C \sqcup D$ to **every** node
- But, **termination is not guaranteed**
 - ▶ E.g., consider $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{Human \sqsubseteq \exists hasMother.Human\} \\ \mathcal{A} &= \{umberto:Human\}\end{aligned}$$



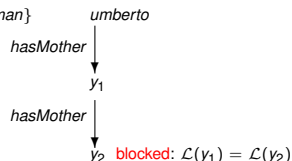
Node Blocking

- When creating new node, check ancestors for equal label set
- If such a node is found, new node is **blocked**
- No rule is applied to blocked nodes

$\mathcal{L}(\text{umberto}) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$

$\mathcal{L}(y_1) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$

$\mathcal{L}(y_2) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$



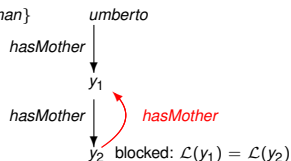
Node Blocking

- When creating new node, check ancestors for equal label set
- If such a node is found, new node is **blocked**
- No rule is applied to blocked nodes

$\mathcal{L}(umberto) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$

$\mathcal{L}(y_1) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$

$\mathcal{L}(y_2) = \{Human, \neg Human \sqcup \exists hasMother.Human, \exists hasMother.Human\}$

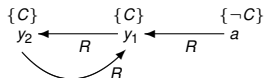
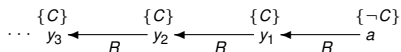


- Block represents **cyclical** model
 - ▶ $\Delta^{\mathcal{I}} = \{umberto, y_1, y_2\}$
 - ▶ $Human^{\mathcal{I}} = \{umberto, y_1, y_2\}$
 - ▶ $hasMother^{\mathcal{I}} = \{\langle umberto, y_1 \rangle, \langle y_1, y_2 \rangle, \langle y_2, y_1 \rangle\}$

Node Blocking: A Note

- With number restrictions ($\mathcal{N}$) and inverse roles ($\mathcal{I}$), some satisfiable concepts have **non-finite** models
 - e.g., testing the satisfiability of $\mathcal{K} = \langle \mathcal{T}, \mathcal{A} \rangle$

$$\begin{aligned}\mathcal{T} &= \{ \top \sqsubseteq \exists R.C, \top \sqsubseteq (\leq 1 R^-) \} \\ \mathcal{A} &= \{ a : \neg C \}\end{aligned}$$



Not a model as $\top \sqsubseteq (\leq 1 R^-)$ violated

DL Reasoning: Advanced Techniques

- More **expressive** DLs
 - ▶ Basic technique can be extended to deal with
 - ★ Role inclusion axioms (role hierarchy)
 - ★ Number restrictions
 - ★ Inverse roles
 - ★ Concrete domains/datatypes
 - ★ etc.
 - ▶ Extend expansion rules and use **more sophisticated blocking strategy**

DL Reasoning: Optimised Implementations

- Naive implementation can lead to effective **non-termination**
 - ▶ 10 GCIs with 10 nodes $\mapsto 2^{100}$ different possible expansions
- Modern systems include **MANY** optimisations
- Optimised **classification** (compute partial ordering)
 - ▶ Enhanced traversal (exploits information from previous tests)
 - ▶ Use structural information to select classification order
- Optimised **satisfiability**/subsumption testing
 - ▶ Normalisation and simplification of concepts
 - ▶ Absorption (simplification) of axioms
 - ▶ Dependency directed backtracking
 - ▶ Caching of satisfiability results and (partial) models
 - ▶ Heuristic ordering of propositional and modal expansion
 - ▶ ...

Acknowledgements

Thanks to various people from whom I “borrowed” material:

- Frank van Harmelen
- Ian Horrocks
- Peter F. Patel-Schneider
- Sean Bechhofer
- Grigoris Antoniou
- Volker Haarslev

And thanks to all the people from whom they borrowed it .